Chapitre 0 — Avant tout | Partie C — Déplacer une lettre | leçon 0.29 sur 153 | difficulté 1 / 10

0.29. Revenir au départ avec va_a

Le 0.28, plus une ligne : va_a(x, y, …, 0, 0) ramène la lettre au coin, sans compter les pas du retour.

1. Ce qu'il faut comprendre

C'est le 0.28, plus une ligne : le retour à (0, 0).

Ce qui est nouveau ici : rien qu'une autre arrivée. Pour revenir, on ne calcule pas « -10 et -5 » : on dit simplement **où** : `0, 0`.

Le chemin du retour suit la même règle : en diagonale tant que les deux axes avancent, de (10, 5) à (5, 0), puis tout droit jusqu'en (0, 0).

Les fonctions de cette leçon — et le tuto de chacune

- `#include <va_a>` → 0.27.3. La fonction va_a() — aller jusqu'à une case
- `#include <ALPHABET>` → 0.1.4. ALPHABET — les lettres de la police
- `#include <nombre>` → 0.8.3. La fonction nombre() — écrire un nombre

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **affectation (=)** — « x = 5; » : la boîte x prend la valeur 5. Ce n'est pas une égalité : c'est un rangement.
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **coordonnées** — Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **octet** — Un nombre de 0 à 255 : la plus petite boîte de la mémoire. Une cartouche en a 32 768.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **uint8_t** — Le genre de nombre de cette console : un entier de 0 à 255 (un octet). Au-delà, il repart à 0.
- **variable** — Une boîte avec un nom, dans la mémoire, qui retient un nombre qu'on peut lire et changer.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`#include <va_a>  // mène une tuile jusqu'à une case`	Demande à la console « va_a » (ce que c'est : mène une tuile jusqu'à une case). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « mène une tuile jusqu'à une case ».)
`#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « les lettres de la police : ALPHABET[0] est le A ».)

`#include <nombre>  // écrit un nombre en chiffres`	Demande à la console « nombre » (ce que c'est : écrit un nombre en chiffres). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « écrit un nombre en chiffres ».)
`uint8_t x = 0;  // la COLONNE de la lettre`	Une variable : une boîte appelée x, qui retient un nombre de 0 à 255, et qui commence à 0. (Le commentaire le dit : « la COLONNE de la lettre ».)
`uint8_t y = 0;  // la LIGNE de la lettre`	Une variable : une boîte appelée y, qui retient un nombre de 0 à 255, et qui commence à 0. (Le commentaire le dit : « la LIGNE de la lettre ».)
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`va_a(x, y, ALPHABET[0], 10, 5);  // l'aller (le 0.28) : jusqu'en (10, 5)`	Appel de la fonction va_a de la console, avec les arguments x, y, ALPHABET[0], 10, 5. Ce qu'elle fait : va jusqu'à la case (versColonne, versLigne), un pas tous les quarts de seconde : en diagonale, puis tout droit. Seule sur sa ligne : va_a(x, y, ALPHABET[0], 10, 5); range l'arrivée dans x et y. (Le commentaire le dit : « l'aller (le 0.28) : jusqu'en (10, 5) ».)
`// La ligne ajoutée : l'arrivée est le coin (0, 0).`	Un commentaire : une phrase pour celui qui lit (« La ligne ajoutée : l'arrivée est le coin (0, 0). »). La console l'ignore.
`//  de (10, 5), en diagonale jusqu'en (5, 0), puis tout droit jusqu'en (0, 0)`	Un commentaire : une phrase pour celui qui lit (« de (10, 5), en diagonale jusqu'en (5, 0), puis tout droit jusqu'en (0, 0) »). La console l'ignore.
`va_a(x, y, ALPHABET[0], 0, 0);`	Appel de la fonction va_a de la console, avec les arguments x, y, ALPHABET[0], 0, 0. Ce qu'elle fait : va jusqu'à la case (versColonne, versLigne), un pas tous les quarts de seconde : en diagonale, puis tout droit. Seule sur sa ligne : va_a(x, y, ALPHABET[0], 10, 5); range l'arrivée dans x et y.
`nombre(0, 17, x);  // 000`	Appel de la fonction nombre de la console, avec les arguments 0, 17, x. Ce qu'elle fait : écrit un nombre calculé, en base dix. Le 4e argument dit combien de chiffres (3 par défaut). Un 5e est la TAILLE, sans limite : nombre(2, 2, score, 3, 4) écrit le score quatre fois plus grand. (Le commentaire le dit : « 000 ».)
`nombre(4, 17, y);  // 000`	Appel de la fonction nombre de la console, avec les arguments 4, 17, y. Ce qu'elle fait : écrit un nombre calculé, en base dix. Le 4e argument dit combien de chiffres (3 par défaut). Un 5e est la TAILLE, sans limite : nombre(2, 2, score, 3, 4) écrit le score quatre fois plus grand. (Le commentaire le dit : « 000 ».)
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image();`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1176 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        62 o  ← la console, à l'allumage  — la mise en route de la console
  va_a()          176 o  ← main()  — une fonction fournie par le langage
  main()           83 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces    12 o  ← EffacerCases, EcrireNombre  — attend de pouvoir écrire une case
  AdresseCase      10 o  ← poser(), effacer()  — trouve l'adresse d'une case de l'écran
  EffacerCases     12 o  ← effacer()  — efface des cases
  EcrireNombre     33 o  ← nombre()  — écrit un nombre en chiffres
  CopierTuiles     18 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles          704 o  ← CopierTuiles  — la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, Diviser, Reste, Multiplier, DecalerGauche,
DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert,
Notes
Fonctions du langage appelées :
  poser()   × 2    26 o sur place, dans va_a()  + AdresseCase (10 o)
  nombre()  × 2    20 o sur place, dans main()  + EcrireNombre (33 o)
  effacer() × 1    15 o sur place, dans va_a()  + AdresseCase (10 o), EffacerCases (12 o)
  image()   × 2     6 o sur place, dans va_a(), main()  + AttendreImage (28 o)
Tes fonctions appelées :
  va_a()    × 2    44 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <va_a>       // mène une tuile jusqu'à une case
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
#include <nombre>     // écrit un nombre en chiffres
```

2. UNE VARIABLE — X

```
uint8_t x = 0;        // la COLONNE de la lettre
```

3. UNE VARIABLE — Y

```
uint8_t y = 0;        // la LIGNE de la lettre
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
  va_a(x, y, ALPHABET[0], 10, 5);   // l'aller (le 0.28) : jusqu'en (10, 5)

  // La ligne ajoutée : l'arrivée est le coin (0, 0).
  //   de (10, 5), en diagonale jusqu'en (5, 0), puis tout droit jusqu'en (0, 0)
  va_a(x, y, ALPHABET[0], 0, 0);

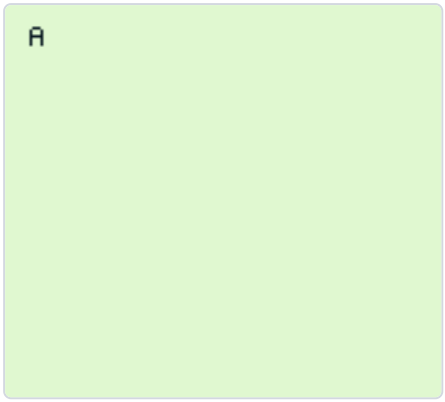
  nombre(0, 17, x);     // 000
  nombre(4, 17, y);     // 000

  while (true) {
    image();
  }
}
```

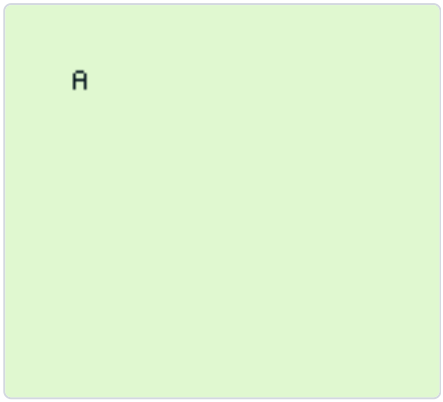
3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

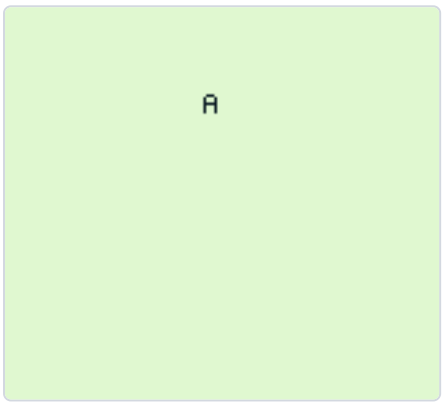
la toute première image — la console vient de démarrer — image 1



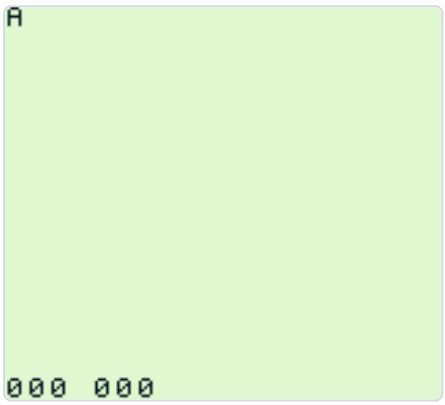
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

> Le A va jusqu'en (10, 5), puis revient au coin (0, 0), en biais puis tout droit.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ l'aller jusqu'en (10, 5)
- ✓ le retour en biais : (7, 2), puis tout droit : (3, 0)
- ✓ revenu en (0, 0) ; x et y valent 0

Ensuite : **0.29.1. Revenir au départ avec va_a — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++